

Sentence Memorability Experiment

Testing Hypotheses 1 and 2: Role of Lexical Condition and Grammatical Voice in
Recognition Memory

Team:- Metamatics

April 06, 2026

Contents

Introduction	2
Background	2
Experimental Procedure	3
Our Hypotheses	3
Our Key Measure: Corrected Memorability	3
Setup and Data Loading	3
Libraries We Need	3
Reading the Log Files	4
Data Cleaning	4
Step 1: Remove Practice Trials	4
Step 2: Parse Stimulus Codes	5
Step 3: Convert Data Types	6
Step 4: Reconstruct Trials	6
Participant Exclusion	7
Validation-Based Quality Control	7
Computing Corrected Memorability	11
Deriving Original Voice	11
Mapping Lexical Conditions	12
Descriptive Statistics by Original Voice	14
Linking Study and Test Trials (WR Accuracy)	15
Computing CM per Condition x Voice (for H1 and H2)	17
Preparing Data for Each Hypothesis	17

Hypothesis 1: Effect of Lexical Condition	18
Descriptive Statistics	18
Normality Testing	19
Q-Q Plots for Normality Assessment	20
Visualization: Distribution of Corrected Memorability	21
Statistical Test: Kruskal-Wallis	21
Post-Hoc Pairwise Comparisons	22
Hypothesis 2: Effect of Grammatical Voice	23
Descriptive Statistics	23
Paired Differences	24
Normality of Differences	24
Q-Q Plot of Paired Differences	25
Visualization: Voice Comparison	25
Statistical Test: Wilcoxon Signed-Rank	28
Reaction Time Analysis	28
Conclusion	30
Summary of Findings	30
Interpretation	31

Introduction

Background

This report was prepared by **Team Metamatics** as part of our research project on sentence memorability. We ran a **continuous recognition experiment** to explore how two factors shape whether people remember a sentence:

1. **Lexical condition** – do the nouns in a sentence have high or low memorability?
2. **Grammatical voice** – is the sentence Active or Passive?

Participants read a stream of simple Subject-Verb-Object (SVO) sentences. Each sentence appeared twice: first in a study phase, then in a test phase. Their job was to spot repeated sentences and to say whether the wording had stayed the same.

Experimental Procedure

- **Study Phase:** A sentence appears on screen for the first time.
- **Test Phase:** The sentence appears again. The participant must:
 - Press **Spacebar** (IR = Immediate Recognition) if they recognise it as a repeat.
 - Press **Yes/No** (WR = Wording Recognition) to judge if the wording has changed.

We built sentences by pairing nouns that differ in memorability:

Code	Subject Noun	Object Noun	Example
HH	High	High	<i>The doctor examined the patient.</i>
HL	High	Low	<i>The teacher read the passage.</i>
LH	Low	High	<i>The device recorded the music.</i>
LL	Low	Low	<i>The item covered the surface.</i>

Our Hypotheses

Hypothesis 1: Corrected memorability will differ across the four noun categories (HH, HL, LH, LL). Sentences made from more memorable nouns should be easier to recognise.

Hypothesis 2: Corrected memorability will differ between Active and Passive voice. We want to find out if voice alone changes how well people remember a sentence.

Our Key Measure: Corrected Memorability

$$CM = \text{Hit Rate} - \text{False Alarm Rate}$$

This simple subtraction removes guessing bias: we take the proportion of correctly spotted repeats and subtract the proportion of new sentences that were wrongly called “old”.

Setup and Data Loading

Libraries We Need

```
# Auto-install anything that is missing
pkgs <- c(
  "dplyr", "tidyr", "ggplot2", "readr", "stringr", "purrr",
  "knitr", "kableExtra", "RColorBrewer", "scales"
)
for (p in pkgs) {
  if (!requireNamespace(p, quietly = TRUE)) {
    install.packages(p, repos = "https://cran.r-project.org")
  }
}
```

```

library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)
library(stringr)
library(purrr)
library(knitr)
library(kableExtra)
library(scales)

# A minimal, modern ggplot theme we will reuse everywhere
theme_set(theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold", size = 15),
    plot.subtitle = element_text(color = "grey40"),
    legend.position = "bottom"
  ))

```

Reading the Log Files

Our dataset lives in **114 separate log files** – one for each participant. Every file records all the events that happened during that person’s session.

```

log_dir <- file.path("Sentence Memorability", "NewLogsAnonymized")
output_dir <- "R_output"
if (!dir.exists(output_dir)) dir.create(output_dir, recursive = TRUE)

log_files <- list.files(log_dir, pattern = "\\\\.log$", full.names = TRUE)
cat(sprintf("Found %d participant log files.\n", length(log_files)))

## Found 114 participant log files.

# Load every file and stack them into a single table
raw_data <- log_files %>%
  map_dfr(~ tryCatch(
    read_csv(., show_col_types = FALSE, col_types = cols(.default = "c")),
    error = function(e) tibble()
  ))

cat(sprintf("Total rows loaded: %s\n", format(nrow(raw_data), big.mark = ",")))

## Total rows loaded: 81,329

```

Data Cleaning

Step 1: Remove Practice Trials

Practice trials let people get used to the task before the real experiment. We drop them so they do not skew our results.

```
# Count and remove practice events
n_practice <- sum(str_starts(raw_data$Event, "Practice") & !is.na(raw_data$Event))
raw_data <- raw_data %>% filter(!str_starts(Event, "Practice") | is.na(Event))

cat(sprintf("Removed %d practice events.\n", n_practice))
```

```
## Removed 5269 practice events.
```

```
cat(sprintf("Remaining rows: %s\n", format(nrow(raw_data), big.mark = ",")))
```

```
## Remaining rows: 76,060
```

```
# Save cleaned data
write_csv(raw_data, file.path(output_dir, "step1_cleaned_data.csv"))
cat("Saved: step1_cleaned_data.csv\n")
```

```
## Saved: step1_cleaned_data.csv
```

Step 2: Parse Stimulus Codes

Every stimulus has a code like "HH_172_A". The last letter is the voice (A = Active, P = Passive), and everything before it is the sentence ID. We split these apart so we can group by condition and voice later.

```
# Parse each stimulus into Core_ID and Voice
parse_stimulus <- function(stim) {
  if (is.na(stim)) {
    return(c(NA_character_, NA_character_))
  }
  parts <- str_split(stim, "_")[[1]]
  n <- length(parts)
  if (n >= 2) {
    return(c(paste(parts[1:(n - 1)], collapse = "_"), parts[n]))
  }
  return(c(stim, NA_character_))
}
```

```
# Apply to all rows - use Stimulus column
stim_col <- raw_data[["Stimulus"]]
if (is.null(stim_col)) {
  # Fallback: try to find a column containing stimulus info
  stim_col <- rep(NA_character_, nrow(raw_data))
  warning("Stimulus column not found - using NA")
}
```

```
parsed <- t(apply(stim_col, parse_stimulus, USE.NAMES = FALSE))
raw_data$Core_ID <- parsed[, 1]
raw_data$Voice <- parsed[, 2]

cat("Stimulus codes parsed successfully.\n")
```

```
## Stimulus codes parsed successfully.
```

```
cat(sprintf("Unique voices found: %s\n", paste(unique(na.omit(raw_data$Voice)), collapse = ", ")))
```

```
## Unique voices found: A, P
```

Step 3: Convert Data Types

All columns arrive as text because we read every file with `col_types = "c"`. Here we turn numbers into actual numbers and flag columns into TRUE/FALSE.

```
# Turn text columns into the right types
# NA in a boolean column means the event did not happen, so we treat it as FALSE
raw_data <- raw_data %>%
  mutate(
    participant_ID = as.integer(participant_ID),
    Timestamp      = as.numeric(Timestamp),
    `Accuracy IR`  = suppressWarnings(as.numeric(`Accuracy IR`)),
    `Accuracy WR`  = suppressWarnings(as.numeric(`Accuracy WR`)),
    Reaction_time_IR = suppressWarnings(as.numeric(Reaction_time_IR)),
    Reaction_time_WR = suppressWarnings(as.numeric(Reaction_time_WR)),
    isTarget        = coalesce(tolower(trimws(isTarget)) == "true", FALSE),
    isValid         = coalesce(tolower(trimws(isValidation)) == "true", FALSE),
    isRepeat        = coalesce(tolower(trimws(isRepeat)) == "true", FALSE)
  ) %>%
  arrange(participant_ID, Timestamp)

cat(sprintf(
  "Data types converted. %d unique participants.\n",
  n_distinct(raw_data$participant_ID)
))
```

```
## Data types converted. 114 unique participants.
```

Step 4: Reconstruct Trials

Each trial starts when a sentence appears on screen and includes every response event until the next sentence. We stitch these rows together into one row per trial.

```
# Assign trial IDs: each "Sentence shown" starts a new trial
raw_data <- raw_data %>%
  group_by(participant_ID) %>%
  mutate(trial_id = cumsum(Event == "Sentence shown")) %>%
  ungroup() %>%
  filter(trial_id > 0)

# Extract metadata from the "Sentence shown" row
sent_rows <- raw_data %>%
  filter(Event == "Sentence shown") %>%
  group_by(participant_ID, trial_id) %>%
  slice(1) %>%
  ungroup() %>%
  select(
```

```

    participant_ID, trial_id, Core_ID, Voice,
    isTarget, isValid, isRepeat
  )

# Aggregate response events within each trial
trial_agg <- raw_data %>%
  group_by(participant_ID, trial_id) %>%
  summarise(
    ir_pressed = any(str_detect(Event, "IR pressed"), na.rm = TRUE),
    wr_pressed = any(str_detect(Event, "WR pressed"), na.rm = TRUE),
    has_validation = any(str_detect(Event, "Validation"), na.rm = TRUE),
    ir_accuracy = max(`Accuracy IR`, na.rm = TRUE),
    wr_accuracy = max(`Accuracy WR`, na.rm = TRUE),
    ir_rt = suppressWarnings(min(Reaction_time_IR, na.rm = TRUE)),
    wr_rt = suppressWarnings(min(Reaction_time_WR, na.rm = TRUE)),
    .groups = "drop"
  ) %>%
  mutate(across(
    c(ir_accuracy, wr_accuracy, ir_rt, wr_rt),
    ~ ifelse(is.infinite(.), NA_real_, .)
  ))

# Combine metadata + responses
trials <- sent_rows %>%
  inner_join(trial_agg, by = c("participant_ID", "trial_id")) %>%
  mutate(
    is_validation = isValid | has_validation,
    is_target = isTarget,
    is_repeat = isRepeat
  )

cat(sprintf(
  "Reconstructed %s trials from %d participants.\n",
  format(nrow(trials), big.mark = ","), n_distinct(trials$participant_ID)
))

## Reconstructed 25,308 trials from 114 participants.

# Save trial-level data
write_csv(trials, file.path(output_dir, "step2_trials.csv"))
cat("Saved: step2_trials.csv\n")

```

```
## Saved: step2_trials.csv
```

Participant Exclusion

Validation-Based Quality Control

Our experiment has built-in **validation filler sentences**. These are easy catch trials that tell us whether a participant was actually paying attention. We use a simple rule to decide who stays and who gets dropped.

Exclusion rule – a participant passes only if:

$$\text{correct_ir} > \frac{\text{wrong_ir}}{2} + \text{missed_ir}$$

```
val_trials <- trials %>% filter(is_validation)

validation_summary <- val_trials %>%
  group_by(participant_ID) %>%
  summarise(
    correct_ir = sum(is_repeat == TRUE & ir_pressed == TRUE, na.rm = TRUE),
    wrong_ir = sum(is_repeat == FALSE & ir_pressed == TRUE, na.rm = TRUE),
    missed_ir = sum(is_repeat == TRUE & ir_pressed == FALSE, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(passes = correct_ir > (wrong_ir / 2) + missed_ir)

valid_ids <- validation_summary %>%
  filter(passes) %>%
  pull(participant_ID)
excluded_ids <- validation_summary %>%
  filter(!passes) %>%
  pull(participant_ID)

# Display validation results table
validation_summary %>%
  mutate(Status = ifelse(passes, "KEPT", "EXCLUDED")) %>%
  select(participant_ID, correct_ir, wrong_ir, missed_ir, Status) %>%
  kable(
    caption = "Participant Validation Results",
    col.names = c("Participant", "Correct IR", "Wrong IR", "Missed IR", "Status"),
    booktabs = TRUE
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    latex_options = c("striped", "scale_down", "hold_position"),
    full_width = FALSE, position = "center"
  )
```

Table 2: Participant Validation Results

Participant	Correct IR	Wrong IR	Missed IR	Status
232	30	16	0	KEPT
235	29	22	1	KEPT
236	30	15	0	KEPT
241	29	24	1	KEPT
242	27	3	3	KEPT
243	30	7	0	KEPT
245	21	19	9	KEPT
246	27	7	3	KEPT
247	26	4	4	KEPT
249	29	26	1	KEPT

251	30	44	0	KEPT
252	30	10	0	KEPT
253	30	15	0	KEPT
254	28	32	2	KEPT
256	28	20	2	KEPT
257	26	14	4	KEPT
258	24	8	6	KEPT
259	30	29	0	KEPT
260	28	8	2	KEPT
261	28	26	2	KEPT
262	27	6	3	KEPT
263	28	17	2	KEPT
264	30	14	0	KEPT
266	30	13	0	KEPT
267	30	22	0	KEPT
269	29	5	1	KEPT
270	24	1	6	KEPT
271	30	73	0	EXCLUDED
273	27	6	3	KEPT
276	29	13	1	KEPT
277	30	25	0	KEPT
279	30	10	0	KEPT
280	27	11	3	KEPT
281	29	46	1	KEPT
282	30	26	0	KEPT
283	25	6	5	KEPT
285	30	6	0	KEPT
286	29	13	1	KEPT
289	30	15	0	KEPT
290	29	10	1	KEPT
292	29	43	1	KEPT
293	30	9	0	KEPT
294	29	37	1	KEPT
295	30	5	0	KEPT
296	28	13	2	KEPT
297	30	42	0	KEPT
298	26	10	4	KEPT
299	13	20	17	EXCLUDED
302	28	22	2	KEPT
303	28	6	2	KEPT
304	28	21	2	KEPT
305	30	16	0	KEPT
307	30	21	0	KEPT
308	30	34	0	KEPT
309	21	5	9	KEPT
310	26	11	4	KEPT
311	30	15	0	KEPT
312	29	18	1	KEPT
313	30	48	0	KEPT
314	29	16	1	KEPT

316	30	18	0	KEPT
317	27	27	3	KEPT
318	30	15	0	KEPT
322	27	26	3	KEPT
323	30	7	0	KEPT
325	28	13	2	KEPT
326	29	16	1	KEPT
327	30	12	0	KEPT
328	29	16	1	KEPT
329	28	11	2	KEPT
330	28	26	2	KEPT
331	27	7	3	KEPT
333	30	43	0	KEPT
339	30	38	0	KEPT
340	30	8	0	KEPT
342	30	45	0	KEPT
343	30	33	0	KEPT
344	30	8	0	KEPT
346	27	46	3	KEPT
350	30	9	0	KEPT
351	30	25	0	KEPT
354	28	10	2	KEPT
355	30	8	0	KEPT
356	29	14	1	KEPT
361	30	11	0	KEPT
362	26	12	4	KEPT
363	29	19	1	KEPT
364	26	5	4	KEPT
366	28	26	2	KEPT
367	30	51	0	KEPT
368	30	6	0	KEPT
369	25	14	5	KEPT
370	28	21	2	KEPT
371	25	10	5	KEPT
372	27	9	3	KEPT
373	30	48	0	KEPT
374	27	21	3	KEPT
375	30	9	0	KEPT
377	28	26	2	KEPT
378	29	11	1	KEPT
379	27	13	3	KEPT
380	27	8	3	KEPT
381	30	35	0	KEPT
382	29	13	1	KEPT
383	30	24	0	KEPT
384	29	9	1	KEPT
386	26	7	4	KEPT
390	27	16	3	KEPT
391	30	11	0	KEPT
392	29	26	1	KEPT

395	28	13	2	KEPT
396	30	6	0	KEPT
397	30	6	0	KEPT
398	27	16	3	KEPT

```
cat(sprintf("Total participants:    %d\n", nrow(validation_summary)))

## Total participants:    114

cat(sprintf("Participants excluded: %d %s\n", length(excluded_ids), paste(excluded_ids, collapse = ", ")))

## Participants excluded: 2  271, 299

cat(sprintf("Participants kept:    %d\n", length(valid_ids)))

## Participants kept:    112

# Save validation summary
write_csv(validation_summary, file.path(output_dir, "step3_validation_summary.csv"))
cat("Saved: step3_validation_summary.csv\n")

## Saved: step3_validation_summary.csv

# Filter to valid participants only
trials_valid <- trials %>% filter(participant_ID %in% valid_ids)
```

Computing Corrected Memorability

Deriving Original Voice

When a sentence first appears (study phase), it is in either Active or Passive voice. We call this the **Original Voice**. We grab it from the first non-repeat presentation of each sentence for every participant.

```
# Get Original Voice from first (non-repeat) presentation of each Core_ID
first_pres <- trials_valid %>%
  filter(!is_repeat) %>%
  distinct(participant_ID, Core_ID, .keep_all = TRUE) %>%
  select(participant_ID, Core_ID, Voice) %>%
  rename(Original_Voice = Voice)

# Attach Original Voice to valid trials
trials_valid <- trials_valid %>%
  left_join(first_pres, by = c("participant_ID", "Core_ID"))

cat(sprintf(
  "Original Voice attached. Unique voices: %s\n",
  paste(unique(na.omit(trials_valid$Original_Voice)), collapse = ", ")
))
```

```
## Original Voice attached. Unique voices: A, P
```

Mapping Lexical Conditions

We tag each sentence with its lexical condition based on the prefix of its Core_ID (HH, HVL, LVH, LVL, or HF for fillers).

```
get_condition <- function(core_id) {
  if (is.na(core_id)) {
    return("Other")
  }
  prefix <- str_to_upper(str_extract(core_id, "[A-Za-z]+"))
  mapping <- c("HH" = "HH", "HVL" = "HL", "LVH" = "LH", "LVL" = "LL", "HF" = "Filler")
  result <- mapping[prefix]
  ifelse(is.na(result), "Other", result)
}

trials_valid <- trials_valid %>%
  mutate(label_group = sapply(Core_ID, get_condition))

# Show distribution of conditions
trials_valid %>%
  count(label_group, name = "Trial_Count") %>%
  kable(
    caption = "Number of Trials per Condition",
    col.names = c("Condition", "Number of Trials"),
    booktabs = TRUE
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover"),
    latex_options = c("striped", "hold_position"),
    full_width = FALSE, position = "center"
  )
```

Table 3: Number of Trials per Condition

Condition	Number of Trials
Filler	14112
HH	2688
HL	2688
LH	2688
LL	2688

We now calculate our main dependent variable. For each participant and each Original Voice, we work out:

- **Hit Rate** = proportion of repeated sentences correctly recognised
- **FA Rate** = proportion of new sentences wrongly called “old” (computed from raw IR-pressed events with Accuracy IR == 0)
- **Corrected Memorability** = Hit Rate - FA Rate

```
# Keep only trials with valid Original Voice
voice_trials <- trials_valid %>%
  filter(Original_Voice %in% c("A", "P"))
```

```

# --- Hits: from repeat trials grouped by participant x Original_Voice ---
hit_data <- voice_trials %>%
  group_by(participant_ID, Original_Voice) %>%
  summarise(
    n_old = sum(is_repeat == TRUE, na.rm = TRUE),
    hits = sum(is_repeat == TRUE & ir_pressed == TRUE, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(hit_rate = ifelse(n_old > 0, hits / n_old, NA_real_))

# --- FAs: from raw IR pressed events where Accuracy IR == 0 ---
# (kept participants only, using raw_data not trial-level)
raw_kept <- raw_data %>%
  filter(participant_ID %in% valid_ids)

# All "IR pressed" events
ir_all <- raw_kept %>%
  filter(Event == "IR pressed") %>%
  left_join(first_pres, by = c("participant_ID", "Core_ID"))

# All first-presentation (non-repeat) Sentence shown events (FA denominator)
first_shown <- raw_kept %>%
  filter(Event == "Sentence shown", isRepeat == FALSE) %>%
  left_join(first_pres, by = c("participant_ID", "Core_ID"))

# Compute FA rate per participant x Original_Voice
fa_list <- list()
for (pid in valid_ids) {
  for (voice in c("A", "P")) {
    fa_count <- ir_all %>%
      filter(
        participant_ID == pid,
        Original_Voice == voice,
        `Accuracy IR` == 0
      ) %>%
      nrow()
    nontarget <- first_shown %>%
      filter(
        participant_ID == pid,
        Original_Voice == voice
      ) %>%
      nrow()
    fa_rate <- ifelse(nontarget > 0, fa_count / nontarget, NA_real_)
    fa_list[[length(fa_list) + 1]] <- tibble(
      participant_ID = pid,
      Original_Voice = voice,
      false_alarms = fa_count,
      n_new = nontarget,
      fa_rate = fa_rate
    )
  }
}
fa_df <- bind_rows(fa_list)

```

```

# Merge hits and FAs
mem_df <- hit_data %>%
  full_join(fa_df, by = c("participant_ID", "Original_Voice")) %>%
  mutate(corrected_memorability = hit_rate - fa_rate)

# Quick overview of hit/FA/CM for each voice
cat("\n=== Corrected Memorability per participant x voice ===\n")

##
## === Corrected Memorability per participant x voice ===

for (v in c("A", "P")) {
  sub <- mem_df %>% filter(Original_Voice == v)
  cat(sprintf("\nVoice %s:\n", v))
  cat(sprintf("  Mean Hit Rate:    %.4f\n", mean(sub$hit_rate, na.rm = TRUE)))
  cat(sprintf("  Mean FA Rate:      %.4f\n", mean(sub$fa_rate, na.rm = TRUE)))
  cat(sprintf("  Mean Corrected M:  %.4f\n", mean(sub$corrected_memorability, na.rm = TRUE)))
}

##
## Voice A:
##   Mean Hit Rate:    0.8937
##   Mean FA Rate:     0.1390
##   Mean Corrected M: 0.7547
##
## Voice P:
##   Mean Hit Rate:    0.8411
##   Mean FA Rate:     0.0450
##   Mean Corrected M: 0.7961

# Save mem_df
write_csv(mem_df, file.path(output_dir, "step4_mem_df.csv"))
cat("Saved: step4_mem_df.csv\n")

## Saved: step4_mem_df.csv

```

Descriptive Statistics by Original Voice

```

# Full descriptive statistics table for each voice
voice_describe <- mem_df %>%
  pivot_longer(
    cols = c(hit_rate, fa_rate, corrected_memorability),
    names_to = "Metric", values_to = "Value"
  ) %>%
  group_by(Original_Voice, Metric) %>%
  summarise(
    Count = sum(!is.na(Value)),
    Mean = round(mean(Value, na.rm = TRUE), 6),
    SD = round(sd(Value, na.rm = TRUE), 6),

```

```

Min = round(min(Value, na.rm = TRUE), 6),
Q25 = round(quantile(Value, 0.25, na.rm = TRUE), 6),
Median = round(median(Value, na.rm = TRUE), 6),
Q75 = round(quantile(Value, 0.75, na.rm = TRUE), 6),
Max = round(max(Value, na.rm = TRUE), 6),
.groups = "drop"
) %>%
mutate(
  Metric = factor(Metric,
    levels = c("hit_rate", "fa_rate", "corrected_memorability"),
    labels = c("Hit Rate", "FA Rate", "Corrected Memorability")
  )
)

voice_describe %>%
  kable(
    caption = "Descriptive Statistics -- Hit Rate, FA Rate, and CM by Original Voice",
    col.names = c("Voice", "Metric", "N", "Mean", "SD", "Min", "Q1", "Median", "Q3", "Max"),
    booktabs = TRUE,
    longtable = TRUE
  ) %>%
  kable_styling(
    latex_options = c("striped", "repeat_header", "scale_down"),
    full_width = FALSE,
    position = "center",
    font_size = 10
  )

```

Table 4: Descriptive Statistics – Hit Rate, FA Rate, and CM by Original Voice

Voice	Metric	N	Mean	SD	Min	Q1	Median	Q3	Max
A	Corrected Memorability	112	0.754696	0.115382	0.402778	0.666898	0.775000	0.842824	0.939815
A	FA Rate	112	0.138988	0.092885	0.008333	0.072917	0.108333	0.183333	0.391667
A	Hit Rate	112	0.893684	0.099049	0.444444	0.851852	0.907407	0.962963	1.000000
P	Corrected Memorability	112	0.796131	0.147908	0.250000	0.708333	0.833333	0.916667	1.000000
P	FA Rate	112	0.045015	0.048157	0.000000	0.000000	0.041667	0.083333	0.208333
P	Hit Rate	112	0.841146	0.148138	0.250000	0.791667	0.875000	0.958333	1.000000

Linking Study and Test Trials (WR Accuracy)

```

# Study Events: First presentation (non-repeat)
study_df <- trials_valid %>%
  filter(!is_repeat) %>%
  distinct(participant_ID, Core_ID, .keep_all = TRUE) %>%
  select(participant_ID, Core_ID, Voice) %>%
  rename(Original_Voice = Voice)

# Test Events: Repeat trials where WR was pressed
test_df <- trials_valid %>%

```

```

filter(is_repeat == TRUE, wr_pressed == TRUE) %>%
distinct(participant_ID, Core_ID, .keep_all = TRUE) %>%
select(participant_ID, Core_ID, Voice, wr_accuracy, wr_rt) %>%
rename(Test_Voice = Voice, `Accuracy WR` = wr_accuracy, Reaction_time_WR = wr_rt)

# Merge Study and Test
analysis_df <- test_df %>%
  inner_join(study_df, by = c("participant_ID", "Core_ID")) %>%
  mutate(Condition = ifelse(Original_Voice == Test_Voice, "Identical", "Transformed")) %>%
  filter(Original_Voice %in% c("A", "P"))

# Merge corrected memorability from mem_df
analysis_df <- analysis_df %>%
  left_join(
    mem_df %>% select(participant_ID, Original_Voice, corrected_memorability, hit_rate, fa_rate),
    by = c("participant_ID", "Original_Voice")
  )

cat(sprintf("\nTotal matched test trials: %d\n", nrow(analysis_df)))

##
## Total matched test trials: 7666

cat(sprintf("Unique participants: %d\n", n_distinct(analysis_df$participant_ID)))

## Unique participants: 112

cat(sprintf("\nOriginal Voice counts:\n"))

##
## Original Voice counts:

print(table(analysis_df$Original_Voice))

##
##      A      P
## 5405 2261

cat(sprintf("\nCondition counts:\n"))

##
## Condition counts:

print(table(analysis_df$Condition))

##
##      Identical Transformed
##          5489          2177

```

```
# Save analysis_df
write_csv(analysis_df, file.path(output_dir, "step4_analysis_df.csv"))
cat("Saved: step4_analysis_df.csv\n")
```

```
## Saved: step4_analysis_df.csv
```

Computing CM per Condition x Voice (for H1 and H2)

```
# Filter to target conditions only (exclude fillers and validation)
target <- trials_valid %>%
  filter(
    !is_validation,
    label_group %in% c("HH", "HL", "LH", "LL"),
    Voice %in% c("A", "P")
  )

sentence_mem <- target %>%
  group_by(participant_ID, label_group, Voice) %>%
  summarise(
    n_old = sum(is_repeat == TRUE, na.rm = TRUE),
    n_new = sum(is_repeat == FALSE, na.rm = TRUE),
    hits = sum(is_repeat == TRUE & ir_pressed == TRUE, na.rm = TRUE),
    false_alarms = sum(is_repeat == FALSE & ir_pressed == TRUE, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    hit_rate = ifelse(n_old > 0, hits / n_old, NA_real_),
    false_alarm_rate = ifelse(n_new > 0, false_alarms / n_new, NA_real_),
    corrected_memorability = hit_rate - false_alarm_rate
  )

# Save detailed memorability data
write_csv(sentence_mem, file.path(output_dir, "step4_sentence_memorability.csv"))
cat(sprintf("sentence_mem: %d rows\n", nrow(sentence_mem)))
```

```
## sentence_mem: 896 rows
```

Preparing Data for Each Hypothesis

```
COND_ORDER <- c("HH", "HL", "LH", "LL")

# H1: average across voice so we get one score per condition per participant
mem_h1 <- sentence_mem %>%
  group_by(participant_ID, label_group) %>%
  summarise(
    corrected_memorability = mean(corrected_memorability, na.rm = TRUE),
    .groups = "drop"
  ) %>%
```

```

mutate(label_group = factor(label_group, levels = COND_ORDER))

# H2: average across condition so we get one score per voice per participant
mem_h2 <- sentence_mem %>%
  group_by(participant_ID, Voice) %>%
  summarise(
    corrected_memorability = mean(corrected_memorability, na.rm = TRUE),
    .groups = "drop"
  )

write_csv(mem_h1, file.path(output_dir, "step5_mem_h1.csv"))
write_csv(mem_h2, file.path(output_dir, "step5_mem_h2.csv"))
cat(sprintf("H1 data: %d rows | H2 data: %d rows\n", nrow(mem_h1), nrow(mem_h2)))

```

```
## H1 data: 448 rows | H2 data: 224 rows
```

Hypothesis 1: Effect of Lexical Condition

Descriptive Statistics

Here is a quick look at the average corrected memorability in each of the four noun-memorability groups.

```

h1_desc <- mem_h1 %>%
  group_by(label_group) %>%
  summarise(
    N = n(),
    Mean = round(mean(corrected_memorability, na.rm = TRUE), 3),
    Median = round(median(corrected_memorability, na.rm = TRUE), 3),
    SD = round(sd(corrected_memorability, na.rm = TRUE), 3),
    Min = round(min(corrected_memorability, na.rm = TRUE), 3),
    Max = round(max(corrected_memorability, na.rm = TRUE), 3),
    .groups = "drop"
  )

h1_desc %>%
  kable(
    caption = "Descriptive Statistics -- Corrected Memorability by Lexical Condition",
    col.names = c("Condition", "N", "Mean", "Median", "SD", "Min", "Max"),
    booktabs = TRUE
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "bordered"),
    latex_options = c("striped", "hold_position"),
    full_width = FALSE, position = "center"
  )

```

Table 5: Descriptive Statistics – Corrected Memorability by Lexical Condition

Condition	N	Mean	Median	SD	Min	Max
HH	112	0.850	0.917	0.163	0.333	1
HL	112	0.852	0.917	0.165	0.167	1
LH	112	0.836	0.917	0.175	0.083	1
LL	112	0.794	0.833	0.183	0.083	1

Normality Testing

We need to know if the data are roughly normal before picking a test. The Shapiro-Wilk test checks this for each group. If the p-value is below 0.05, we cannot assume normality.

```
# Safe wrapper for shapiro.test (handles small samples)
safe_shapiro <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) < 3 || length(x) > 5000) {
    return(data.frame(W = NA_real_, p_value = NA_real_))
  }
  st <- shapiro.test(x)
  data.frame(W = round(as.numeric(st$statistic), 4), p_value = as.numeric(st$p.value))
}

sw_h1 <- mem_h1 %>%
  group_by(label_group) %>%
  summarise(
    N = n(), sw = list(safe_shapiro(corrected_memorability)),
    .groups = "drop"
  ) %>%
  unnest(sw) %>%
  mutate(
    Normal = case_when(is.na(p_value) ~ "N/A", p_value > 0.05 ~ "Yes", TRUE ~ "No"),
    p_display = case_when(
      is.na(p_value) ~ "N/A", p_value < 0.001 ~ "< 0.001",
      TRUE ~ as.character(round(p_value, 4))
    )
  )

sw_h1 %>%
  select(label_group, N, W, p_display, Normal) %>%
  kable(
    caption = "Shapiro-Wilk Normality Test per Lexical Condition",
    col.names = c("Condition", "N", "W", "p-value", "Normal?"),
    booktabs = TRUE
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover"),
    latex_options = c("striped", "hold_position"),
    full_width = FALSE, position = "center"
  )
```

Table 6: Shapiro-Wilk Normality Test per Lexical Condition

Condition	N	W	p-value	Normal?
HH	112	0.8367	< 0.001	No
HL	112	0.8122	< 0.001	No
LH	112	0.8162	< 0.001	No
LL	112	0.8849	< 0.001	No

Q-Q Plots for Normality Assessment

Another way to check normality visually. If the dots sit close to the straight line, the data are roughly normal.

```
p_qq_h1 <- ggplot(mem_h1, aes(sample = corrected_memorability)) +
  stat_qq(colour = "#6C5CE7", alpha = 0.6, size = 2) +
  stat_qq_line(colour = "#E17055", linewidth = 0.8) +
  facet_wrap(~label_group, scales = "free") +
  labs(
    title = "Q-Q Plots -- Corrected Memorability by Lexical Condition",
    x = "Theoretical Quantiles", y = "Sample Quantiles"
  )
p_qq_h1
```

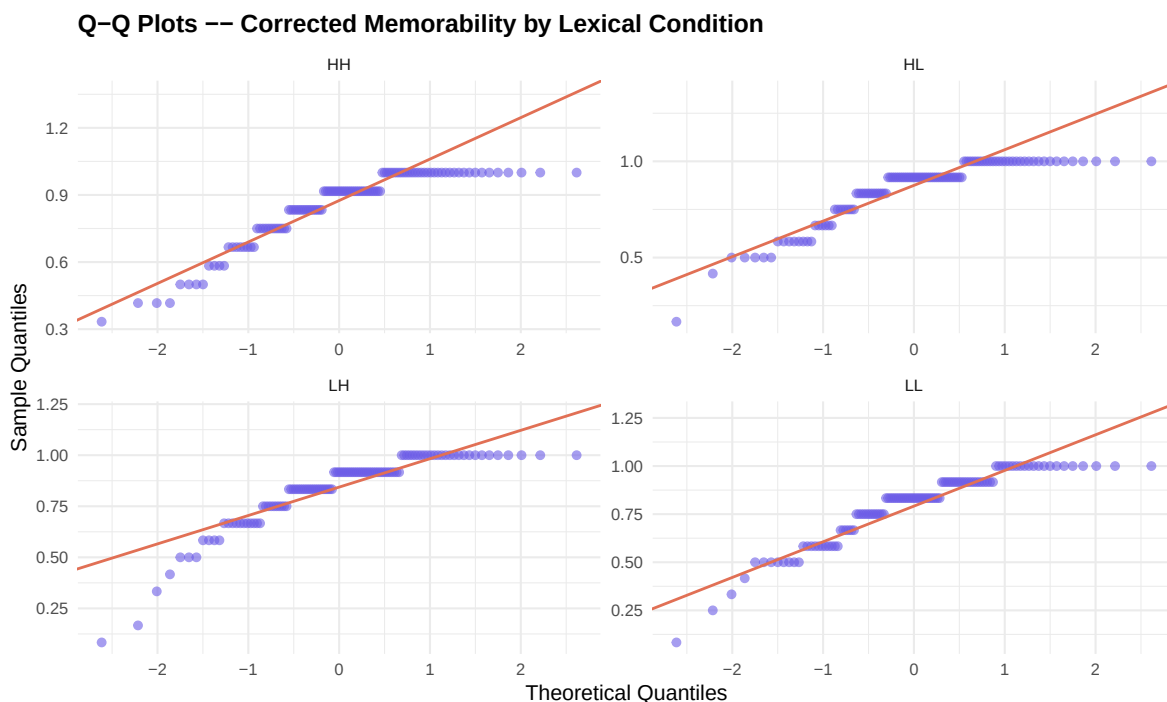


Figure 1: Q-Q Plots of Corrected Memorability by Lexical Condition.

```
ggsave(file.path(output_dir, "H1_QQ_plots.png"), p_qq_h1, width = 10, height = 8, dpi = 150)
cat("Saved: H1_QQ_plots.png\n")
```

```
## Saved: H1_QQ_plots.png
```

Visualization: Distribution of Corrected Memorability

The plot below combines a violin shape (showing the distribution) with a box plot and individual data points. Wider parts of the violin mean more participants scored around that level.

```
p_h1 <- ggplot(mem_h1, aes(
  x = label_group, y = corrected_memorability,
  fill = label_group
)) +
  geom_violin(alpha = 0.5, colour = NA, width = 0.9) +
  geom_boxplot(width = 0.15, outlier.alpha = 0.3, fill = "white") +
  geom_jitter(width = 0.08, alpha = 0.4, size = 1.8, color = "grey30") +
  scale_fill_manual(values = PALETTE_4) +
  labs(
    title = "Corrected Memorability by Lexical Condition",
    subtitle = "Hypothesis 1: Does noun memorability affect sentence recognition?",
    x = "Lexical Condition", y = "Corrected Memorability (Hit Rate - FA Rate)"
  ) +
  theme(legend.position = "none")

p_h1
```

```
ggsave(file.path(output_dir, "H1_violin_box.png"), p_h1, width = 10, height = 6, dpi = 150)
cat("Saved: H1_violin_box.png\n")
```

```
## Saved: H1_violin_box.png
```

Statistical Test: Kruskal-Wallis

Since some groups show departures from normality, we use the **Kruskal-Wallis test** — the non-parametric equivalent of one-way ANOVA. This tests whether at least two of the four conditions differ significantly.

```
kw_test <- kruskal.test(corrected_memorability ~ label_group, data = mem_h1)

# Effect size: Epsilon-squared
n_total <- nrow(mem_h1)
k <- length(COND_ORDER)
epsilon_sq <- round((as.numeric(kw_test$statistic) - k + 1) / (n_total - k), 4)

tibble(
  Test = "Kruskal-Wallis",
  Statistic = round(as.numeric(kw_test$statistic), 3),
  df = as.numeric(kw_test$parameter),
  `p-value` = round(as.numeric(kw_test$p.value), 4),
```

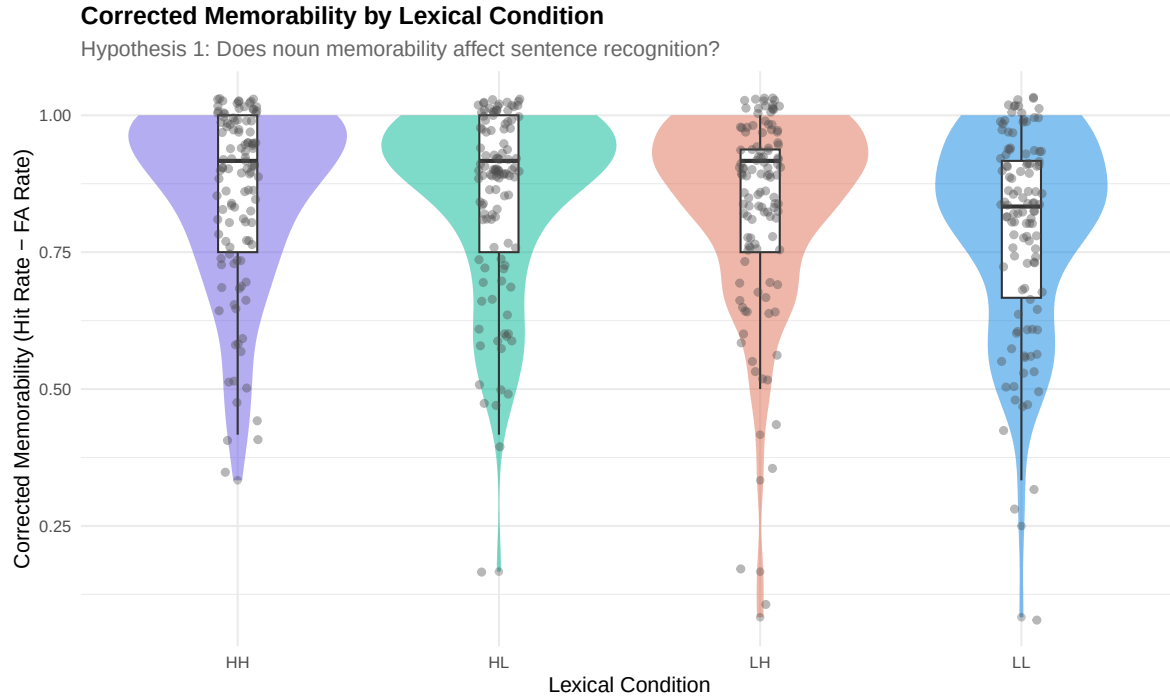


Figure 2: Corrected Memorability across Lexical Conditions. Each dot represents one participant's mean score.

```
`Epsilon2` = epsilon_sq,
Decision   = ifelse(as.numeric(kw_test$p.value) < 0.05, "Reject H0", "Retain H0")
)%>%
kable(caption = "Kruskal-Wallis Test -- H1", booktabs = TRUE) %>%
kable_styling(
  bootstrap_options = c("striped", "hover", "bordered"),
  latex_options = c("striped", "hold_position"),
  full_width = FALSE, position = "center"
)
```

Table 7: Kruskal-Wallis Test – H1

Test	Statistic	df	p-value	Epsilon2	Decision
Kruskal-Wallis	10.753	3	0.0131	0.0175	Reject H0

Post-Hoc Pairwise Comparisons

```
if (as.numeric(kw_test$p.value) < 0.05) {
  pairs <- combn(COND_ORDER, 2, simplify = FALSE)
  posthoc <- map_dfr(pairs, function(pr) {
    x1 <- mem_h1 %>%
      filter(label_group == pr[1]) %>%
      pull(corrected_memorability)
```

```

x2 <- mem_h1 %>%
  filter(label_group == pr[2]) %>%
  pull(corrected_memorability)
wt <- wilcox.test(x1, x2, exact = FALSE)
tibble(
  Pair = paste(pr[1], "vs", pr[2]),
  p_raw = round(as.numeric(wt$p.value), 4),
  p_Bonferroni = round(min(as.numeric(wt$p.value) * length(pairs), 1), 4),
  Significant = ifelse(p_Bonferroni < 0.05, "Yes *", "No")
)
})

posthoc %>%
  kable(
    caption = "Post-Hoc Pairwise Wilcoxon Tests (Bonferroni corrected)",
    col.names = c("Comparison", "p (raw)", "p (Bonferroni)", "Significant?"),
    booktabs = TRUE
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover"),
    latex_options = c("striped", "hold_position"),
    full_width = FALSE, position = "center"
  )
} else {
  cat("Kruskal-Wallis was not significant -- no follow-up comparisons needed.\n")
}

```

Table 8: Post-Hoc Pairwise Wilcoxon Tests (Bonferroni corrected)

Comparison	p (raw)	p (Bonferroni)	Significant?
HH vs HL	0.9645	1.0000	No
HH vs LH	0.4352	1.0000	No
HH vs LL	0.0072	0.0432	Yes *
HL vs LH	0.3855	1.0000	No
HL vs LL	0.0043	0.0258	Yes *
LH vs LL	0.0342	0.2049	No

Hypothesis 2: Effect of Grammatical Voice

Descriptive Statistics

```

h2_desc <- mem_h2 %>%
  group_by(Voice) %>%
  summarise(
    N = n(),
    Mean = round(mean(corrected_memorability, na.rm = TRUE), 3),

```

```

Median = round(median(corrected_memorability, na.rm = TRUE), 3),
SD = round(sd(corrected_memorability, na.rm = TRUE), 3),
.groups = "drop"
) %>%
mutate(Voice = c(A = "Active", P = "Passive")[Voice])

h2_desc %>%
kable(
caption = "Descriptive Statistics -- CM by Grammatical Voice",
col.names = c("Voice", "N", "Mean", "Median", "SD"),
booktabs = TRUE
) %>%
kable_styling(
bootstrap_options = c("striped", "hover", "bordered"),
latex_options = c("striped", "hold_position"),
full_width = FALSE, position = "center"
)

```

Table 9: Descriptive Statistics – CM by Grammatical Voice

Voice	N	Mean	Median	SD
Active	112	0.827	0.875	0.151
Passive	112	0.839	0.875	0.160

Paired Differences

Since every participant has a score for both Active and Passive, we can work out each person's difference (Active minus Passive) and test whether that difference is significantly different from zero.

```

h2_wide <- mem_h2 %>%
pivot_wider(names_from = Voice, values_from = corrected_memorability) %>%
drop_na(A, P) %>%
mutate(diff = A - P)

cat(sprintf("Paired participants: %d\n", nrow(h2_wide)))

```

```
## Paired participants: 112
```

```
cat(sprintf("Mean difference (A - P): %.4f\n", mean(h2_wide$diff)))
```

```
## Mean difference (A - P): -0.0112
```

Normality of Differences

```

sw_diff <- shapiro.test(h2_wide$diff)

tibble(
Test = "Shapiro-Wilk (on differences)",

```

```

W = round(as.numeric(sw_diff$statistic), 4),
p_value = round(as.numeric(sw_diff$p.value), 4),
Normal = ifelse(as.numeric(sw_diff$p.value) > 0.05, "Yes", "No"),
`Recommended Test` = ifelse(as.numeric(sw_diff$p.value) > 0.05, "Paired t-test", "Wilcoxon signed-rank")
) %>%
kable(caption = "Normality Test on Paired Differences (A - P)", booktabs = TRUE) %>%
kable_styling(
  bootstrap_options = c("striped", "hover"),
  latex_options = c("striped", "scale_down", "hold_position"),
  full_width = FALSE, position = "center"
)

```

Table 10: Normality Test on Paired Differences (A - P)

Test	W	p_value	Normal	Recommended Test
Shapiro-Wilk (on differences)	0.9639	0.0041	No	Wilcoxon signed-rank

Q-Q Plot of Paired Differences

```

p_qq_h2 <- ggplot(h2_wide, aes(sample = diff)) +
  stat_qq(colour = "#6C5CE7", alpha = 0.6, size = 2) +
  stat_qq_line(colour = "#E17055", linewidth = 0.8) +
  labs(
    title = "Q-Q Plot -- Paired Differences (Active - Passive)",
    x = "Theoretical Quantiles", y = "Sample Quantiles"
  )

p_qq_h2

```

```

ggsave(file.path(output_dir, "H2_QQ_plot.png"), p_qq_h2, width = 8, height = 6, dpi = 150)
cat("Saved: H2_QQ_plot.png\n")

```

Saved: H2_QQ_plot.png

Visualization: Voice Comparison

```

mem_h2_plot <- mem_h2 %>%
  mutate(Voice_label = c(A = "Active", P = "Passive")[Voice])

p_h2 <- ggplot(mem_h2_plot, aes(
  x = Voice_label, y = corrected_memorability,
  fill = Voice_label
)) +
  geom_violin(alpha = 0.5, colour = NA, width = 0.8) +
  geom_boxplot(width = 0.15, outlier.alpha = 0.3, fill = "white") +
  geom_jitter(width = 0.08, alpha = 0.4, size = 1.8, color = "grey30") +

```

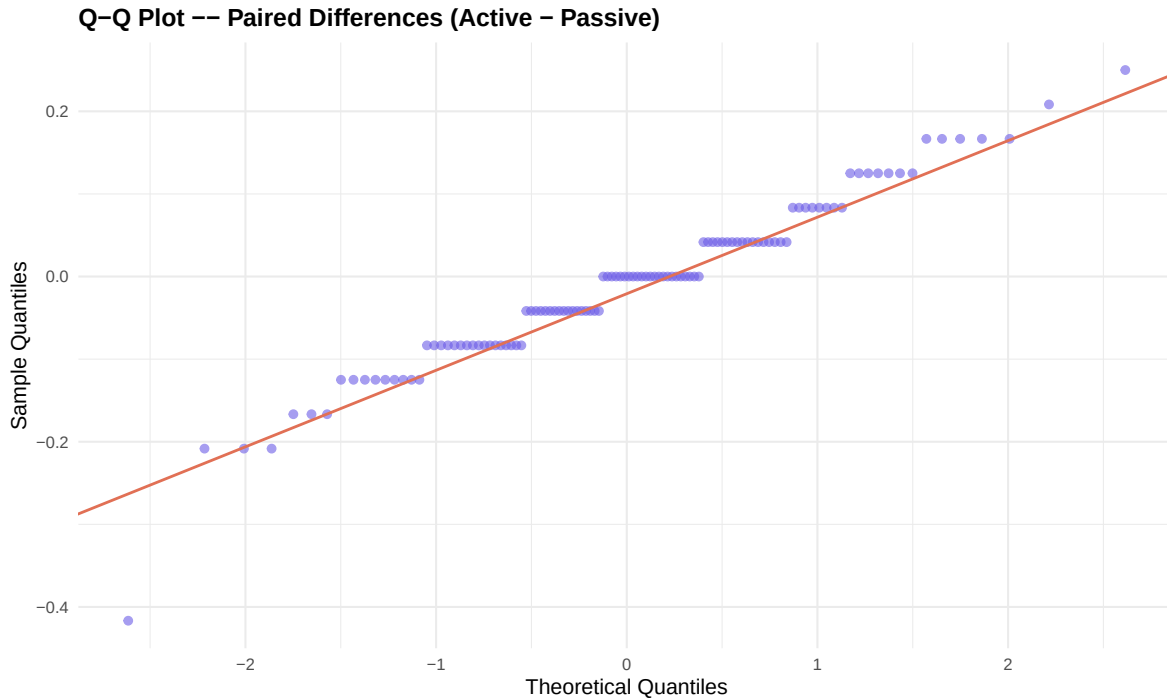


Figure 3: Q-Q Plot of paired differences (Active - Passive). Departures from the line suggest non-normality.

```
scale_fill_manual(values = PALETTE_2) +
labs(
  title = "Corrected Memorability by Grammatical Voice",
  subtitle = "Hypothesis 2: Does Active vs Passive voice affect recognition?",
  x = "Voice", y = "Corrected Memorability"
) +
theme(legend.position = "none")

p_h2

ggsave(file.path(output_dir, "H2_violin_box.png"), p_h2, width = 8, height = 6, dpi = 150)
cat("Saved: H2_violin_box.png\n")
```

Saved: H2_violin_box.png

```
p_hist <- ggplot(h2_wide, aes(x = diff)) +
  geom_histogram(bins = 20, fill = "#6C5CE7", colour = "white", alpha = 0.8) +
  geom_vline(xintercept = 0, colour = "red", linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = mean(h2_wide$diff), colour = "#E17055", linewidth = 1.2) +
  labs(
    title = "Distribution of Paired Differences (Active - Passive)",
    subtitle = "Orange line = observed mean difference",
    x = "Active - Passive CM", y = "Count"
  )

p_hist
```

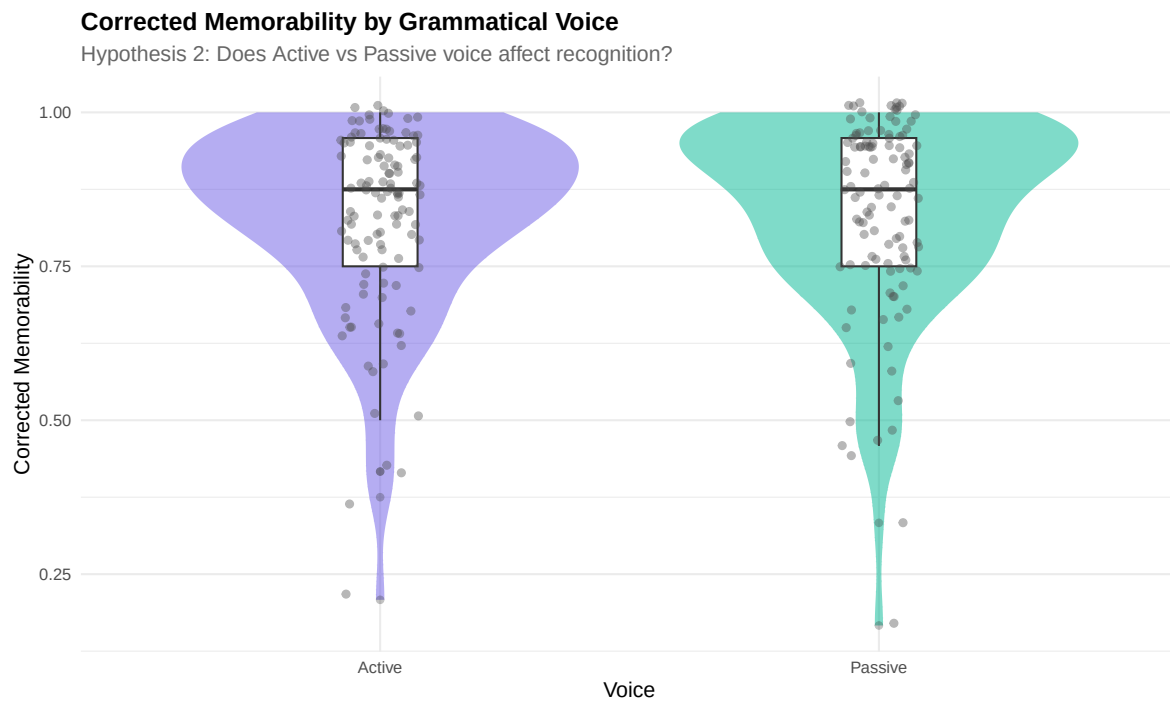


Figure 4: Corrected Memorability by Grammatical Voice.

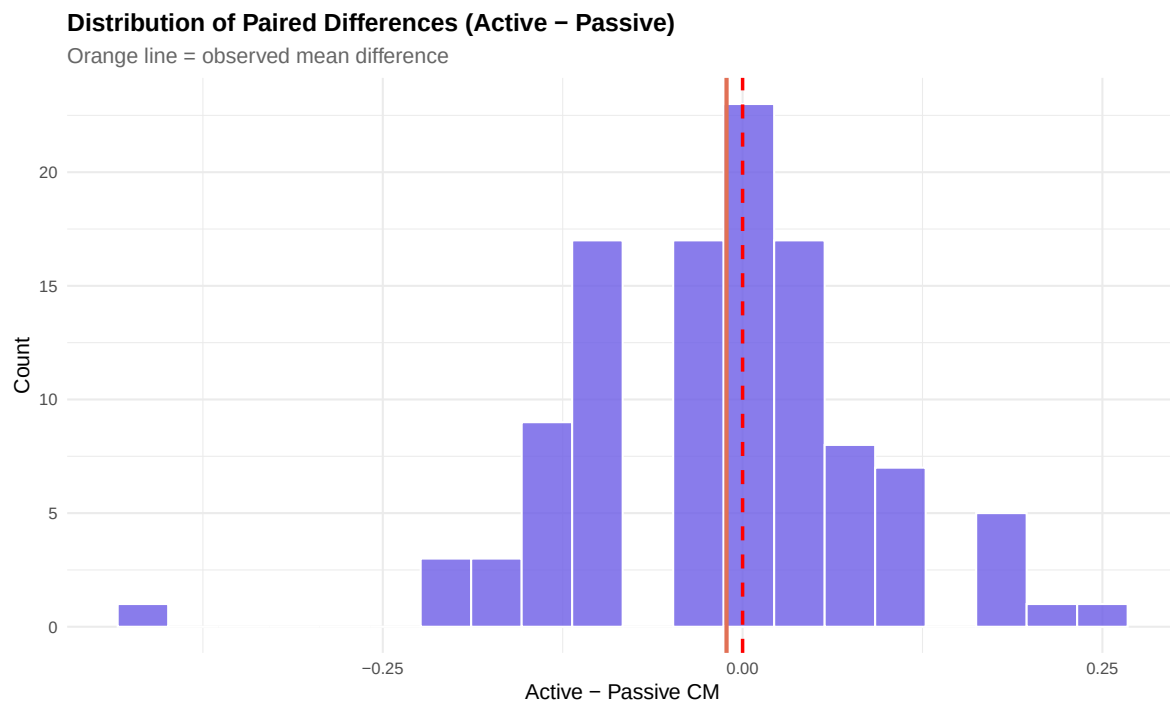


Figure 5: Distribution of paired differences (Active – Passive). Red line = zero, orange line = observed mean.

```
ggsave(file.path(output_dir, "H2_histogram_diff.png"), p_hist, width = 8, height = 5, dpi = 150)
cat("Saved: H2_histogram_diff.png\n")
```

```
## Saved: H2_histogram_diff.png
```

Statistical Test: Wilcoxon Signed-Rank

```
w_test <- wilcox.test(h2_wide$A, h2_wide$P, paired = TRUE, exact = FALSE)

tibble(
  Test      = "Wilcoxon Signed-Rank (paired)",
  W         = round(as.numeric(w_test$statistic), 3),
  `p-value` = round(as.numeric(w_test$p.value), 4),
  Decision  = ifelse(as.numeric(w_test$p.value) < 0.05, "Reject H0", "Retain H0")
) %>%
kable(caption = "Wilcoxon Signed-Rank Test -- H2", booktabs = TRUE) %>%
kable_styling(
  bootstrap_options = c("striped", "hover", "bordered"),
  latex_options = c("striped", "hold_position"),
  full_width = FALSE, position = "center"
)
```

Table 11: Wilcoxon Signed-Rank Test – H2

Test	W	p-value	Decision
Wilcoxon Signed-Rank (paired)	1827.5	0.2928	Retain H0

Reaction Time Analysis

We also looked at how fast people responded when they correctly identified a repeated sentence. The y-axis starts at zero so the scale gives an honest picture.

```
rt_data <- target %>%
  filter(is_repeat, ir_pressed, !is.na(ir_rt), ir_rt > 0) %>%
  mutate(
    label_group = factor(label_group, levels = COND_ORDER),
    Voice_label = ifelse(Voice == "A", "Active", "Passive")
  )

p_rt1 <- ggplot(rt_data, aes(x = label_group, y = ir_rt, fill = label_group)) +
  geom_violin(alpha = 0.45, colour = NA) +
  geom_boxplot(width = 0.15, outlier.alpha = 0.3, fill = "white") +
  scale_fill_manual(values = PALETTE_4) +
  scale_y_continuous(limits = c(0, NA), labels = comma) +
  labs(
```

```

title = "IR Reaction Time by Lexical Condition",
x = "Condition", y = "Reaction Time (ms)"
) +
theme(legend.position = "none")

```

p_rt1

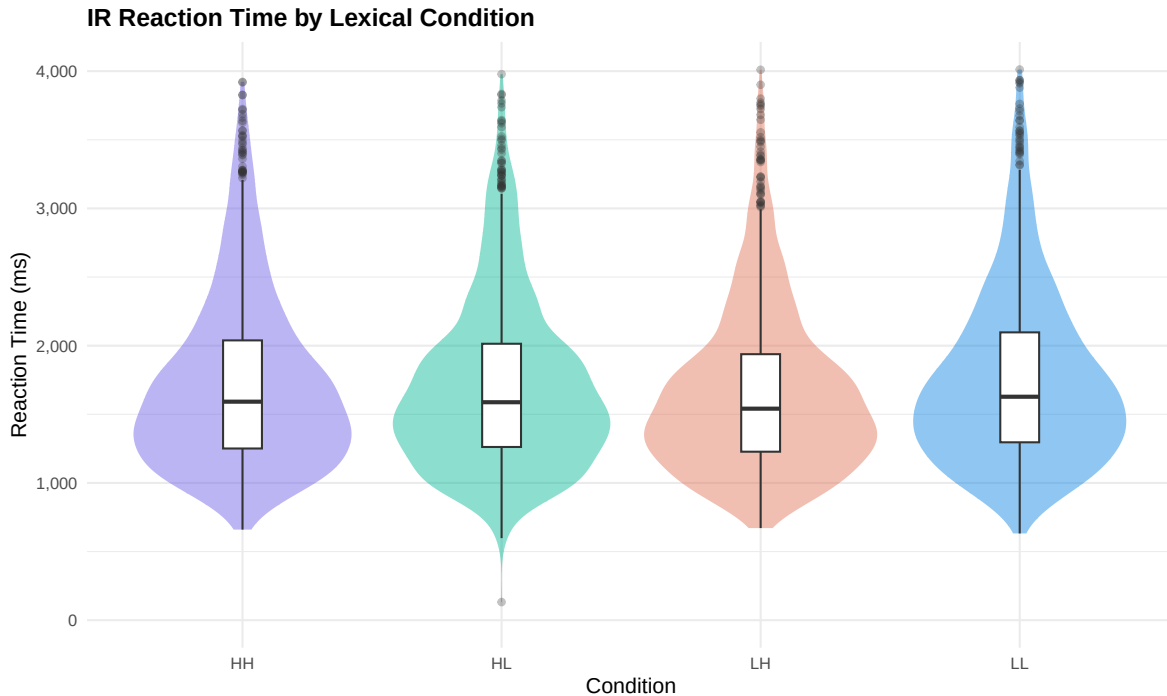


Figure 6: IR Reaction Time by Lexical Condition (y-axis from 0).

```

ggsave(file.path(output_dir, "RT_by_condition.png"), p_rt1, width = 10, height = 6, dpi = 150)

```

```

p_rt2 <- ggplot(rt_data, aes(x = Voice_label, y = ir_rt, fill = Voice_label)) +
  geom_violin(alpha = 0.45, colour = NA) +
  geom_boxplot(width = 0.15, outlier.alpha = 0.3, fill = "white") +
  scale_fill_manual(values = PALETTE_2) +
  scale_y_continuous(limits = c(0, NA), labels = comma) +
  labs(
    title = "IR Reaction Time by Grammatical Voice",
    x = "Voice", y = "Reaction Time (ms)"
  ) +
  theme(legend.position = "none")

```

p_rt2

```

ggsave(file.path(output_dir, "RT_by_voice.png"), p_rt2, width = 8, height = 6, dpi = 150)

```

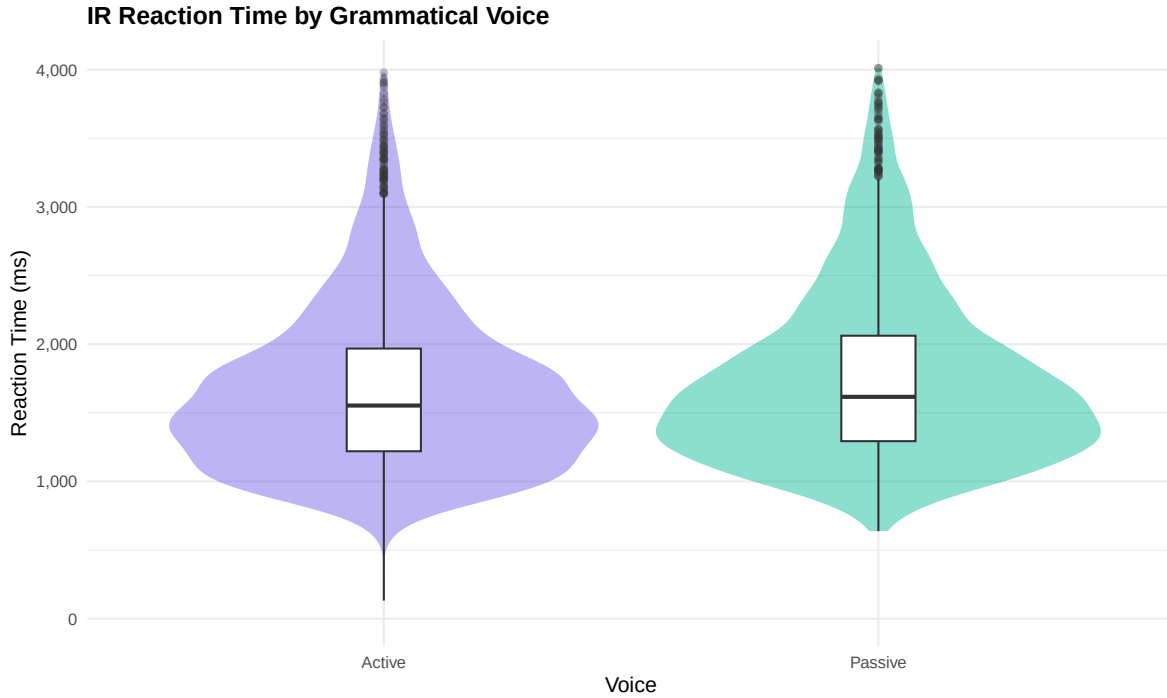


Figure 7: IR Reaction Time by Voice (y-axis from 0).

Conclusion

Summary of Findings

```
tibble(
  Hypothesis = c("H1: Lexical Condition", "H2: Grammatical Voice"),
  Test = c("Kruskal-Wallis", "Wilcoxon Signed-Rank"),
  Statistic = c(round(as.numeric(kw_test$statistic), 3), round(as.numeric(w_test$statistic), 3)),
  `p-value` = c(round(as.numeric(kw_test$p.value), 4), round(as.numeric(w_test$p.value), 4)),
  Result = c(
    ifelse(as.numeric(kw_test$p.value) < 0.05, "Significant *", "Not significant"),
    ifelse(as.numeric(w_test$p.value) < 0.05, "Significant *", "Not significant")
  )
) %>%
kable(caption = "Summary of All Hypothesis Tests", booktabs = TRUE) %>%
kable_styling(
  bootstrap_options = c("striped", "hover", "bordered"),
  latex_options = c("striped", "hold_position"),
  full_width = FALSE, position = "center"
)
```

Table 12: Summary of All Hypothesis Tests

Hypothesis	Test	Statistic	p-value	Result
H1: Lexical Condition	Kruskal-Wallis	10.753	0.0131	Significant *

Interpretation

```
# ----- Hypothesis 1 conclusion -----
kw_p <- as.numeric(kw_test$p.value)

if (kw_p < 0.05) {
  cat("**Hypothesis 1 -- Lexical Condition:**\n\n")
  cat(sprintf("The Kruskal-Wallis test was **significant** (p = %.4f, p < 0.05).\n", kw_p))
  cat("We therefore **reject the null hypothesis (H0)** and conclude that\n")
  cat("corrected memorability **does differ** across at least two of the four\n")
  cat("lexical conditions (HH, HL, LH, LL). In other words, the memorability\n")
  cat("of the nouns in a sentence appears to affect how well people recognise\n")
  cat("that sentence.\n\n")
  # report significant post-hoc pairs if the posthoc object exists
  if (exists("posthoc")) {
    sig_pairs <- posthoc %>% filter(Significant == "Yes *") %>% pull(Pair)
    if (length(sig_pairs) > 0) {
      cat(sprintf("The post-hoc pairwise Wilcoxon tests (with Bonferroni correction) identified\n"))
      cat(sprintf("the following significant pairs: **%s**.\n", paste(sig_pairs, collapse = ", ")))
    } else {
      cat("However, after Bonferroni correction none of the individual pairwise\n")
      cat("comparisons reached significance.\n\n")
    }
  }
} else {
  cat("**Hypothesis 1 -- Lexical Condition:**\n\n")
  cat(sprintf("The Kruskal-Wallis test was **not significant** (p = %.4f, p >= 0.05).\n", kw_p))
  cat("We therefore **retain the null hypothesis (H0)** and conclude that there is\n")
  cat("no statistically significant difference in corrected memorability across the\n")
  cat("four lexical conditions.\n\n")
}
```

Hypothesis 1 – Lexical Condition:

The Kruskal-Wallis test was **significant** ($p = 0.0131$, $p < 0.05$). We therefore **reject the null hypothesis (H0)** and conclude that corrected memorability **does differ** across at least two of the four lexical conditions (HH, HL, LH, LL). In other words, the memorability of the nouns in a sentence appears to affect how well people recognise that sentence.

The post-hoc pairwise Wilcoxon tests (with Bonferroni correction) identified the following significant pairs: **HH vs LL, HL vs LL**.

```
# ----- Hypothesis 2 conclusion -----
w_p <- as.numeric(w_test$p.value)

if (w_p < 0.05) {
  cat("**Hypothesis 2 -- Grammatical Voice:**\n\n")
  cat(sprintf("The Wilcoxon signed-rank test was **significant** (p = %.4f, p < 0.05).\n", w_p))
  cat("We therefore **reject the null hypothesis (H0)** and conclude that there **is**\n")
  cat("a statistically significant paired difference in corrected memorability between\n")
}
```

```

    cat("Active and Passive sentences.\n\n")
  } else {
    cat("**Hypothesis 2 -- Grammatical Voice:**\n\n")
    cat(sprintf("The Wilcoxon signed-rank test was **not significant** (p = %.4f, p >= 0.05).\n", w_p))
    cat("We therefore **retain the null hypothesis (H0)**. There is no statistically\n")
    cat("significant paired difference in corrected memorability between Active and\n")
    cat("Passive sentences.\n\n")
  }
}

```

Hypothesis 2 – Grammatical Voice:

The Wilcoxon signed-rank test was **not significant** ($p = 0.2928$, $p \geq 0.05$). We therefore **retain the null hypothesis (H0)**. There is no statistically significant paired difference in corrected memorability between Active and Passive sentences.

```

cat("All findings should be read together with the effect sizes, Q-Q plots,\n")

```

All findings should be read together with the effect sizes, Q-Q plots,

```

cat("and the distribution plots shown earlier in this report.\n")

```

and the distribution plots shown earlier in this report.
